

MAOS —— 多 Agent 协作运行时

领域无关的编排内核 + 可核验的运行证据 ——
换域零改动，每一步都能被外人重放。

```
python3 scripts/make_evidence.py # ① 跑 7 个场景 + RAG 对照，落成 evidence/  
python3 scripts/verify.py       # ② 八项证据逐项重放校验  
                                → RESULT: 8/8 PASS, exit=0
```

①② 两条缺一不可、顺序不能换。`*.db` 不入 git，直接跑 ② 会报「缺数据库」并退出 2 —— 这是设计行为。

1069	8/8	8	11	13 / 4	Apache-2.0
pytest 全绿	证据核验项	证据束 / 58 个证据文件	Agent Identity (10 个可派单)	Skill / ToolPort	开源许可

三条诊断我们没有绕开，每一条都指向一页具体的、可核验的落点。

评委诊断	落在	台上跑这条 → 看到什么
没有可执行制品和运行证据	P11	python3 scripts/make_evidence.py && python3 scripts/verify.py RESULT: 8/8 PASS
现实业务锚点不足	P3 + P10	python3 run.py --scenario 6 制造售后退款域纵切，不是软件域自证式 demo
「所有 Agent 都回复完成」≠ 业务成功	P10	python3 run.py --scenario 7 Plan 终态 FAILED · biz_status=compensated · settled 观察 0 条

这一页只承诺「指得出落点」，不承诺「已全部解决」
第二条、第三条有代码有证据；第一条要评委自己跑了才算数 —— 所以我们把命令印在封面上，而不是把结论印在封面上。

一条真实形态的售后退款诉求，从三处来源进来，走完受理、裁定、核算、放行、支付、通知——**每一步都往库里落一个真实业务对象。**

客户报修一批轴承，说货有质量问题，要求退款 6800 元。诉求同时来自工单系统、客服聊天记录、客户自拍照片三处。

谁	干了什么	留下了什么（落库对象）
受理 Agent	把三处诉求去重聚合成一个 case	refund_case (submitted)
Manager	规划 DAG —— 与软件交付域是同一个 Manager，零改动	plan + 5 个 task
政策 Agent	按下单时锁定的政策版本裁定退款资格	policy_decision
财务 Agent	核算退款金额并写财务分录（过第六道闸）	finance_entry
支付 Agent	发起退款并观察网关回执	refund_request + payment_observation

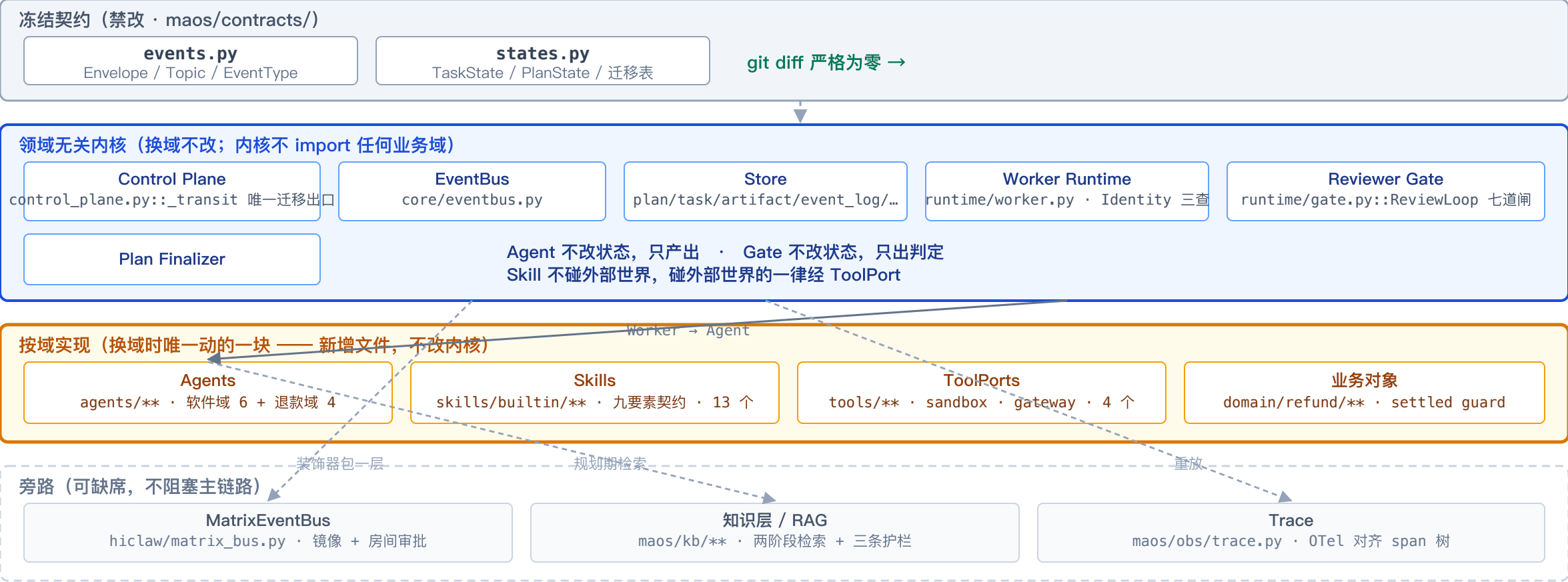
为什么是退款域

它有 MAOS 需要的一切：多源输入、政策裁定、金额核算、**外部权威事实**（钱到没到账）、以及一条走不通时的补偿路径。软件自证式 demo 给不出最后两样。

数据口径（先说清楚）

政策与历史案例是按行业惯例构造的合成数据，不是某家企业的真实政策；支付网关是对齐公开规范的模拟实现。完整口径见 P13。

四块：冻结契约、领域无关内核、按域实现、旁路 —— 换域时只有第三块动。



读法: **FROZEN** 一块在退款域上线前后 git diff 严格为零; **KERNEL** 一块不是零, 但改动全部是领域无关的判据 (第六/第七道闸、重规划口径), 不 import 任何业务域 —— 数字与读法见 P12。

全系统**只有一个出口**能改状态，非法迁移抛异常；产物过**七道闸**，任何一道有 blocker 就返工。

```
# maos/core/control_plane.py::_transit
# 内部：唯一的状态迁移出口
def _transit(self, task, dst, *, event_id="", ...):
    reason = assert_transition(task["state"], dst)
    # ↑ 非法迁移抛异常，不静默改写
    # ↓ 每一次迁移都落一行 event_log
```

PENDING → DISPATCHED → RUNNING → AWAITING_REVIEW
→ DONE | BLOCKED | REWORK | FAILED

业务状态不进 Task 状态机

`compensated` / `settled` 是 `refund_case` 自己的字段，不是 Task 状态 —— 上退款域没加新状态、没加新迁移（见 P12）。

#	闸	判什么
1	schema	产物形状
2	acceptance	验收证据（真跑的 test_report）
3	security	受保护路径
4	evidence	证据完整性
5	compensation	补偿可干跑
6	finance	财务凭据（任务级 + plan 级）
7	gateway	网关回执四象限

第六、七道闸是退款域上线时新增的 —— 但仍领域无关

两道闸都不 import 业务域，只认数据形状。任一道判 blocker，任务回去返工，不是记个警告就放行。

关于「无限重试」：场景 7 只 replan 了 1 次，不是打满上限

撞 `40005` 触发一次 replan 换备用渠道，再撞 `ACQ.SYSTEM_ERROR` 一票否决落人工，全程没有自旋。`MAOS_MAX_REPLAN` 默认 2，这一镜根本没打满。

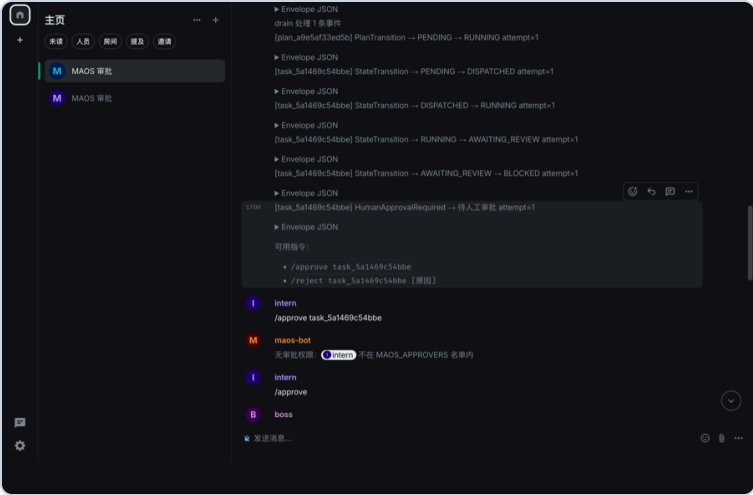
五个概念逐项落到代码位置；事件链的权威记录不在房间里，在 event_log 表。

AgentTeams 概念	MAOS 落点 · 代码位置	状态
Team / 房间	一个 Matrix 房间 = 一条流水线的全部事件；四项配置走环境变量，缺一即降级 matrix_bus.py::MatrixBusConfig · REQUIRED_ENV · from_env	✅ 真房间实测 Synapse v1.159.0
Member / Worker	可插拔 Agent 池：role → Agent 类，投放一个文件即注册 agents/base.py::AGENT_POOL · register · worker.py::Worker.__init__	✅ 已跑通 11 身份 / 10 可派单
事件链 / 消息流	publish() 先走 inner bus，再镜像进房间：一行人话摘要 + 折叠的 Envelope JSON matrix_bus.py::publish · summarize · render_mirror	✅ 真房间实测 两轮 41 条消息
人工介入 / HITL	房间里 /approve、/reject → HumanApprovalQueue.decide()；只认 MAOS_APPROVERS 名单，名单外回「无审批权限」并落一条 event_log matrix_bus.py::parse_approval_command · RoomApprovalBridge.handle_message · _record_denied	✅ 真房间实测 三种各一次
可观测 / 回放	权威记录在 event_log 表；obs/trace.py 转成 OTel 对齐 span 树，verify 第 4 项重放校验「无孤儿、无环、与库逐字节一致」	✅ 已跑通 trace-tree 29/29

当前真实状态（不吹）

真房间已接通。本机自建 Synapse v1.159.0 非加密房（演示房，非生产），两轮 41 条消息，逐字副本见 evidence/room/transcript.md。房间是旁路：连不上就降级 log_only，流水线照跑，等价性有单测。

但房间不是证据。房间消息可被删、被编辑、被折叠——用它当审计链，等于把权威放在聊天软件里。权威记录在 event_log 表（表格第 5 行）。



真房间实拍 evidence/room/02-transitions.png —— 五条状态迁移各自一条消息，不是跑完补一条总结。

Skill 与 ToolPort 都是九要素声明，投放一个文件即注册；三份目录文档由代码生成，不是手写的。

SkillContract · maos/skills/contract.py::SkillContract	
name + version	注册表主键
purpose	① 用途
input_schema / output_schema	② ③ 入出参
preconditions	④ 前置条件
depends_tools	⑤ 依赖工具
failure_policy + max_retries	⑥ 失败策略（合成一项）
security_boundary	⑦ 安全边界
reuse_note	⑧ 复用说明
owner_roles	⑨ 归属角色

注册：类装饰器，模块被 import 即注册

```
@register_skill
class RefundIntakeSkill(Skill):
    contract = SkillContract(...)
```

—— `maos/skills/registry.py::register_skill`，docstring 明写「模块被 import 即注册」。投放即注册，不改任何注册表文件。

ToolPort · maos/tools/port.py::ToolPort	
name	① 名称（审计行按它归集）
purpose	② 用途
entry	③ 入口（契约与实现不许分家）
params_schema	④ 入参
returns_schema	⑤ 出参
failure_modes	⑥ 失败形态 —— 失败被吞掉等于没有边界
security_boundary	⑦ 安全边界 —— 「agent 干不了什么」的答案
rate_limit	⑧ 限流（空 = 明确声明未设限）
owner	⑨ 归属

```
$ python3 scripts/gen_docs.py --check
3 份文档与代码逐字节一致 exit=0
skill-catalog.md · agent-identity.md · toolport-contract.md
```

扫出 13 个 Skill · 5 个已实现 ToolPort (`git-mcp` 走 MCP stdio) · 11 个 Agent Identity —— 三份目录文档全部由运行时代码生成，不一致即非零退出，文档吹不了牛。

阶段一是硬约束不是打分项，按评委给的字段顺序结构化预过滤；阶段二才是四通道混合召回。

阶段一 · 结构化预过滤（顺序即语义）



为什么「不给租户就返回空」而不是回落成全租户检索
回落是最危险的一种默认值：它在单租户的演示里看不出任何异常，多租户上线当天泄漏。

```
# maos/kb/retriever.py::PREFILTER_FIELDS
#: 阶段一的过滤顺序。**顺序即语义**
PREFILTER_FIELDS = (
    "tenant_id", "biz_type", "channel_id",
    "region", "sku",
    "policy_version", "workflow_version",
)

# maos/kb/retriever.py::prefilter
def prefilter(store, query, *, limit=...):
    """阶段一。返回候选集；tenant_id 缺失一律返回空。

    这是硬约束不是打分项 — 本函数**不产生
    任何分数**，只决定谁有资格进入阶段二。"""
```

阶段二 · 四通道混合召回（这里才打分）

规则编号 精确匹配

错误码 精确匹配

全文 关键词

语义 向量近邻

四路加权融合成一个分 —— maos/kb/retriever.py::retrieve。

RAG 的价值要用**有无对照证明**——两版 DAG 的 diff；而检索结果**只能补任务，不能替代事实或跳审批**。

对照实验 · `evidence/scenario-R5/dag-diff.json`

跑法	规划出的 DAG
关掉检索 baseline	计划里 漏排了财务复核任务 → 被第六道闸拦下，返工
打开检索 proposed	命中历史案例，补上财务复核任务 → 差异就是 RAG 的价值，不靠形容词

场景 6 证明到什么，没证明到什么

场景 6 实测 `candidate_count=3` / `hit_count=3` —— 证明到的是链路通 + 确实召回到东西，证明不到「召回准」。「准」的证据只在 `scenario-R5` 的有无对照实验里。

三条护栏 · `maos/kb/guardrails.py::check_all`（评委原话拆成的可执行断言，各带一条负例测试）

- ① 只增不删

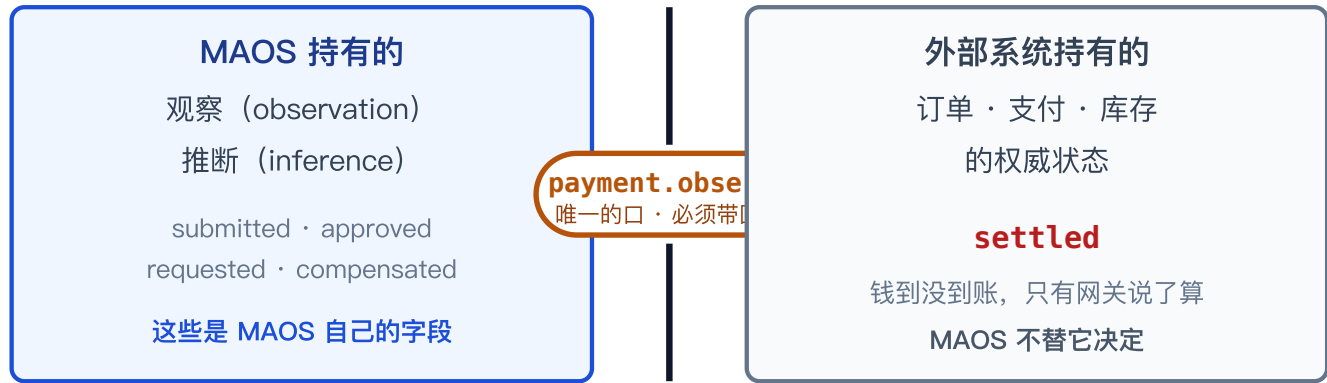
可以往 DAG 里补任务，不能删任何一条既有任务。历史案例里没出现过某一步，不等于这一步不必要——可能只是当时没记全。
- ② 不替代事实

建议任务不许携带订单事实字段。当前订单事实只能从 `order_snapshot` 读，一条都不许从知识层来。
- ③ 不跳审批

不能移除高风险标记、不能把 `effect_risk` 降下来、不能带「免审批」标记。检索到「上次这类单直接放行了」也不行——上次放行是人做的决定。

四个 assert 函数，`check_all` 一次跑完。

MAOS 不持有权威事实。全系统只有 `payment.observe` 写得进 `settled`，且必须同事务附回执 —— 越权抛异常并落证据。



越权不静默失败

抛 `AuthoritativeFactViolation` 并落一条事件。

`payment.observe` 之外的任何 skill 想写 `settled`，或写 `settled` 时没带 observation，两条路都不通。

fail-closed：漏配判据也不放行

加了权威终态却没给回执判据 → 第④道直接拒。宁可写不进去，也不许默认放行 —— 否则「有回执」会被当成默认认真。

```
# maos/domain/refund/guard.py::AUTHORITATIVE_STATES
AUTHORITATIVE_STATES = frozenset({"settled"})

# maos/domain/refund/guard.py::update_biz_status ③
# ③ 权威终态必须有回执。
# 没有回执的 settled 就是把外部状态写死为终态。
_log_violation(store, ...) # ← 落库
raise AuthoritativeFactViolation(...) # ← 抛
```

实况：这个核验器真的抓到过一次绕过

不是「我们从没出过错」，而是 核验器自己抓到了：有一条路径绕过 guard 直接写 SQL，两轨各自测试全绿、合并后才暴露。

根因、为什么两轨都绿、以及那一行修法，全部写在文档里没有删 —— `docs/authoritative-facts.md` §4。

一个能自己发现「边界被绕过」的核验器，比一句「我们划分了边界」有说服力得多。

四个 Agent 全部回复「完成」，而这一单**没有成功**——系统如实这么记了。

```
$ python3 run.py --scenario 7
```

```
[2] 第七道闸认出网关回执: code=40005 retrievable=True
outcome=failed -> disposition=replan_channel
→ 触发 replan: 付款任务换渠道 s7-primary -> s7-backup
幂等键由 (tenant, case) 定, 换渠道不换键, 不会造出第二笔
```

```
task-s7-payment  AWAITING_REVIEW  -> REWORK    [gate_rework]
task-s7-payment  REWORK            -> PENDING   [requeue]
task-s7-payment  BLOCKED           -> FAILED    [human_reject]
```

业务状态 : **compensated** (全程没有经过 settled)
settled 观察: 0 条 — 没问出终态就一条都不该有
补偿记录 : 2 行 ['manual_ticket', 'refund_request_revoked']
补偿事件 : 1 条 CompensationExecuted
Plan 终态 : **FAILED** (主管驳回, 业务确实没成功)
换渠道重试: 1 次 **replan** (40005 触发, ACQ.SYSTEM_ERROR
一票否决, 没有自旋)

这一页是全篇主线

「所有 Agent 都回复完成」≠ 业务成功。四个 Agent 的 `TaskResult` 都是成功返回的, 但网关问不出终态 —— 于是什么都不写, 走补偿 + 转人工。

两条断言把这一页钉死

```
assert case["biz_status"] == "compensated"
assert settled_rows == 0
```

—— `maos/flows/scenario_7.py:692-696`, 注释写着「本场景存在的理由」。跑不出这两条, 测试就红。

两句最容易说过头的话

❌ 重试到上限才转人工 → 只 replan 1 次, 上限是 2, 没打满。

❌ 换子渠道就成功子 → 换完仍未成功, 业务状态收在 `compensated`。

❌ 七个场景都跑成功子 → 场景 7 的 Plan 终态是 **FAILED**, 那正是它要演的。

检索不准顶多说效果一般；无法核验就是零分 —— 所以八项证据每一项都能被外人独立跑一遍。

```
$ python3 scripts/make_evidence.py      # ①
$ python3 scripts/verify.py             # ②

[PASS] hash-integrity      86/86
[PASS] business-ref       35/35
[PASS] authoritative-fact  3/3
[PASS] trace-tree         29/29
[PASS] kb-hit              7/7
[PASS] business-outcome    10/10
[PASS] history-case        1/1
[PASS] cost-attribution    39/39

RESULT: 8/8 PASS    exit=0
证据来源: scenario-1..7, scenario-R5
```

两条命令，缺一不可，顺序不能换

`*.db` 不入 git (`.gitignore` 挡着)，核验器要的是库、不是快照。新克隆直接跑 ② 会报 缺数 数据库: `evidence/scenario-1/maos.db` 并退出 2。这是设计行为，不是故障。

项	失败意味着
hash-integrity	证据被篡改过
business-ref	业务对象引用悬空，锚点是假的
authoritative-fact	权威边界被绕过
trace-tree	事件链有孤儿 / 有环 / 与库不一致
kb-hit	RAG 命中是编的
business-outcome	Agent 完成被当成了业务成功
history-case	知识层被污染（证据不全也进了）
cost-attribution	用量归不到 Run，成本说不清是谁花的

warn 没有被藏起来：12 行 / 3 类

三项会附 `warn:` 行 —— 点名不判负，不改判定但都是真的：

A 类 6 行 产物没有来源事件 D 类 4 行 外部判据来源未审计（说的是入库路径，不是内容真伪）

E 类 2 行 `scenario-7` 有回执但 `biz_status` 不是 `settled` —— 预期内，正是 P10 要演的东西。

两域共用同一份契约、Control Plane、Worker、Gate；换域只换 Skill / ToolPort / 业务对象。

层	软件交付域	制造售后退款域	共用
事件契约	Envelope / EventType	同左，一个字段都没加	✔ 同一份
Task 状态机	PENDING → ... → DONE	同左，没加新状态、没加新迁移	✔ 同一份
Control Plane	唯一迁移持有者、幂等去重	同左	✔ 同一份
Worker Runtime	按 role 取执行者 + Identity 三查	同左	✔ 同一份
Gate	七道闸；看 test_report	七道闸；第六看财务凭据、第七看网关回执	✔ 同一份
HITL 审批	effect_risk=H → BLOCKED	同左，审批人换成退款主管	✔ 同一份
补偿	逆补丁 git_apply(reverse)	撤销 refund_request + 开人工工单	⚠ 机制共用 手段按域
Skill / ToolPort	code.repo-patch · sandbox.*	refund.* · gateway.*	✗ 按域实现 (同一契约)
业务对象 / 角色	补丁集 / 测试报告 · 5 角色	refund_case / finance_entry · 4 角色	✗ 按域实现

上退款域的真实代价（区间 A 实测）

```
$ git diff --shortstat 90251b3 4a70cb0 -- maos/<面>

contracts/      (空, 零改动)
core/           (空, 零改动)
runtime/        1 file, +126/-4   ← 只有 gate.py
agents/ skills/ tools/ domain/
                26 files, +3548/-52 ← 按域实现, 多为纯新增
```

● 不说「内核零改动」这个笼统说法

严格为零的是 maos/contracts/（到当前 HEAD 仍为零）与上退款域区间内的 maos/core/。

全区间到 HEAD: core/ +194/-10、runtime/ +497/-9 —— 这些是后来长出的通用能力（第七道闸、网关码四象限、rework 第三出口、plan 级财务判据），不是退款域改的，且都不 import 业务域。

三条机器守卫钉着这个论证

契约指纹锁 · 内核不识域（AST/import 守卫）· 权威事实边界。

合成数据、公开规范、模拟实现 —— 三者分清楚。这是全场唯一一页主动列自己不能说什么的页。

事实	✅ 只能这么说	❌ 不许这么说
政策与历史案例	按行业惯例构造的合成数据	真实企业政策
支付网关	错误码与异步时序对齐支付宝开放平台公开规范；演示用模拟实现	接入子支付宝
Matrix 房间	真房间实测：本机自建 Synapse v1.159.0 非加密房，两轮 41 条消息、三条审批路径各一次；降级路径等价性另有单测	接的是生产级 Matrix 服务
StorePort / PolarDB	PG 后端已实测跑通；真 PolarDB 实例上 pgvector + zhparser 2.2 装成，24 条真语料三通道召回实测（zhcfg 全文 8/10、向量 10/10）。后端不可用抛 PgBackendUnavailable，仍不回落 sqlite	复赛演示跑在 PolarDB 上
AutoGen	可插拔内核之一，未在复赛演示中启用	基于 AutoGen 构建
replan 换渠道	场景 7 演到了：1 次 replan 换渠道 → 再撞码一票否决落人工	重试到上限才转人工
场景覆盖	run.py 无参跑全部七个场景，含失败路径	七个场景都跑成功子
MCP	5 个 ToolPort 里 1 个（git-mcp）真走 MCP，审计行与本地工具逐字段同形（evidence/scenario-1/trace.json 的 tool:git-mcp）；其余四个刻意没迁，障碍记在 docs/BACKLOG.md	工具层已全面 MCP 化

安全边界：也证明 agent 干不了什么	
代码执行	--network none --read-only --user 1000:1000 --memory 512m --pids-limit 128
降级路径	env 按白名单重建（按名放行，不是按名拦截）
补丁落盘	三重路径校验，任一条不过即拒，不重试不降级
工具 / Skill	Identity 白名单前置校验，越权抛 PermissionDenied
权威事实	越权抛异常 + 落 AuthoritativeFactViolation
密钥	只读环境变量；证据束出口脱敏 + 哨兵串反查，命中即销毁目录并失败
人工授权	只认 MAOS_APPROVERS，名单外落一条 event_log

这是复赛演示实现，不是生产系统

不含客户数据、生产配置，也不做任何不可逆的对外写入 —— 支付网关走对齐官方规范的模拟实现。

克隆、跑测试、跑场景、生成证据、核验 —— 不需要任何 API key，核心零依赖，只要 Python $\geq$ 3.10。

```
git clone <repo> maos && cd maos

python3 -m pytest maos/tests -q      # 860 passed
python3 run.py                       # 场景 1-7, exit=0
python3 run.py --scenario 7          # 单跑失败路径

# 到这里只跑了代码；要看到评委关心的 8/8，
# 还差证据链这两条 ——
python3 scripts/make_evidence.py     # ① 产 8 个证据束
python3 scripts/verify.py           # ② RESULT: 8/8 PASS
```

🔴 最容易漏的一步：① `make_evidence.py`

跳过它直接跑 ②，会看到 缺数据库：evidence/scenario-1/maos.db 并退出 2。这是设计行为 —— *.db 不入 git，核验器要的是库、不是快照。

跑完 `git status` 会有 50 行改动 —— 预期，不是弄坏了

证据文件首行带生成时间与 git sha，每次重跑都会变。但不要用 `git checkout -- evidence/` 去「收拾干净」：json 会还原成旧版本，而 `maos.db` 不入 git 不会跟着还原 —— 新库配旧快照会掉到 3/7 PASS，看上去像证据被伪造，其实只是两边不同步。

全新克隆实测

无任何 API key，clone → ① → ② 约 5 秒到 RESULT: 8/8 PASS，退出码 0。缺省走 Scripted 模式，一行网络都不走，状态迁移序列在任何机器上逐条一致。

评委手上有什么可查

- 8 个证据束 / 58 个证据文件，每个首行都是 `# generated at <ISO8601> from <git sha>`，由脚本写入，不许手写
- 三份目录文档由代码生成 —— `gen_docs.py --check` 不一致即非零退出，文档吹不了牛
- Apache-2.0 开源：代码、文档、证据束、以及本页每一条命令，随仓库一起公开
- 失败与缺口没有删：warn 行原样保留（含 scenario-7 那条「观察到了但没收口」、核验器抓到的那次绕过，全部留在文档里

可核验 = 这份材料的全部说服力

本演示稿的每一页页脚都给了文件行号或一条能当场跑的命令。指不出证据的话，我们没有写。